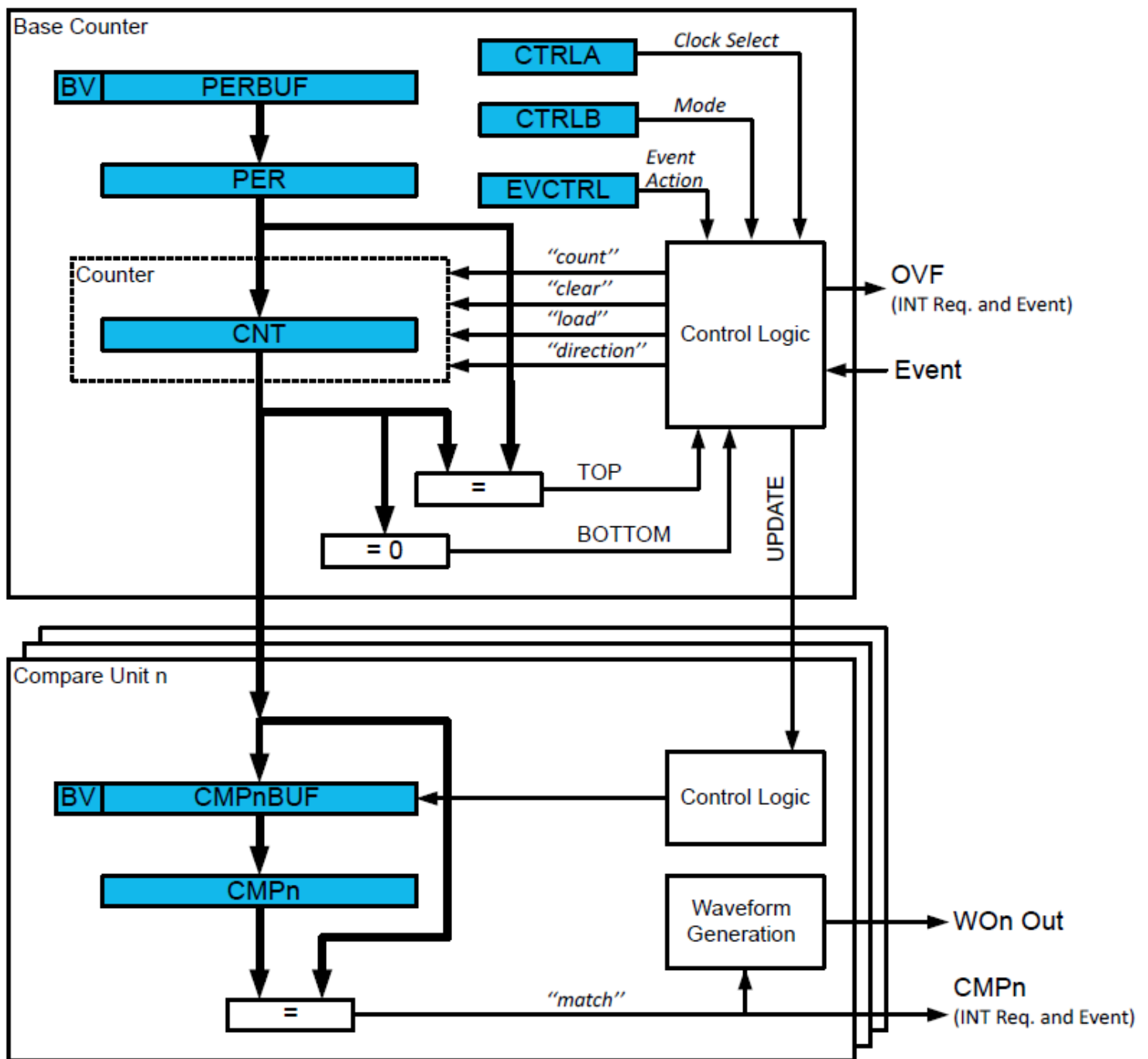


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 6 - Introduction to Timers on AVR DA



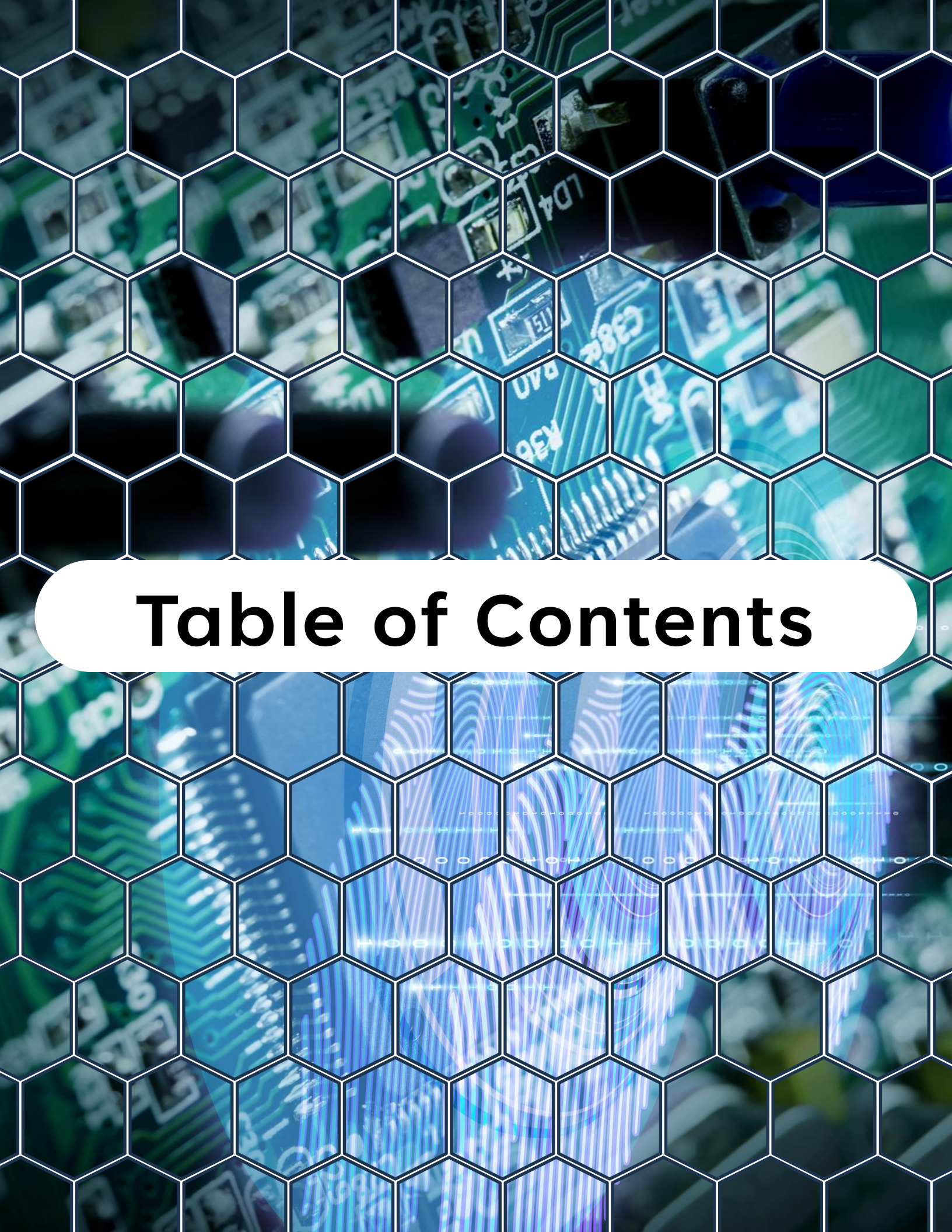


Table of Contents

Table of Contents

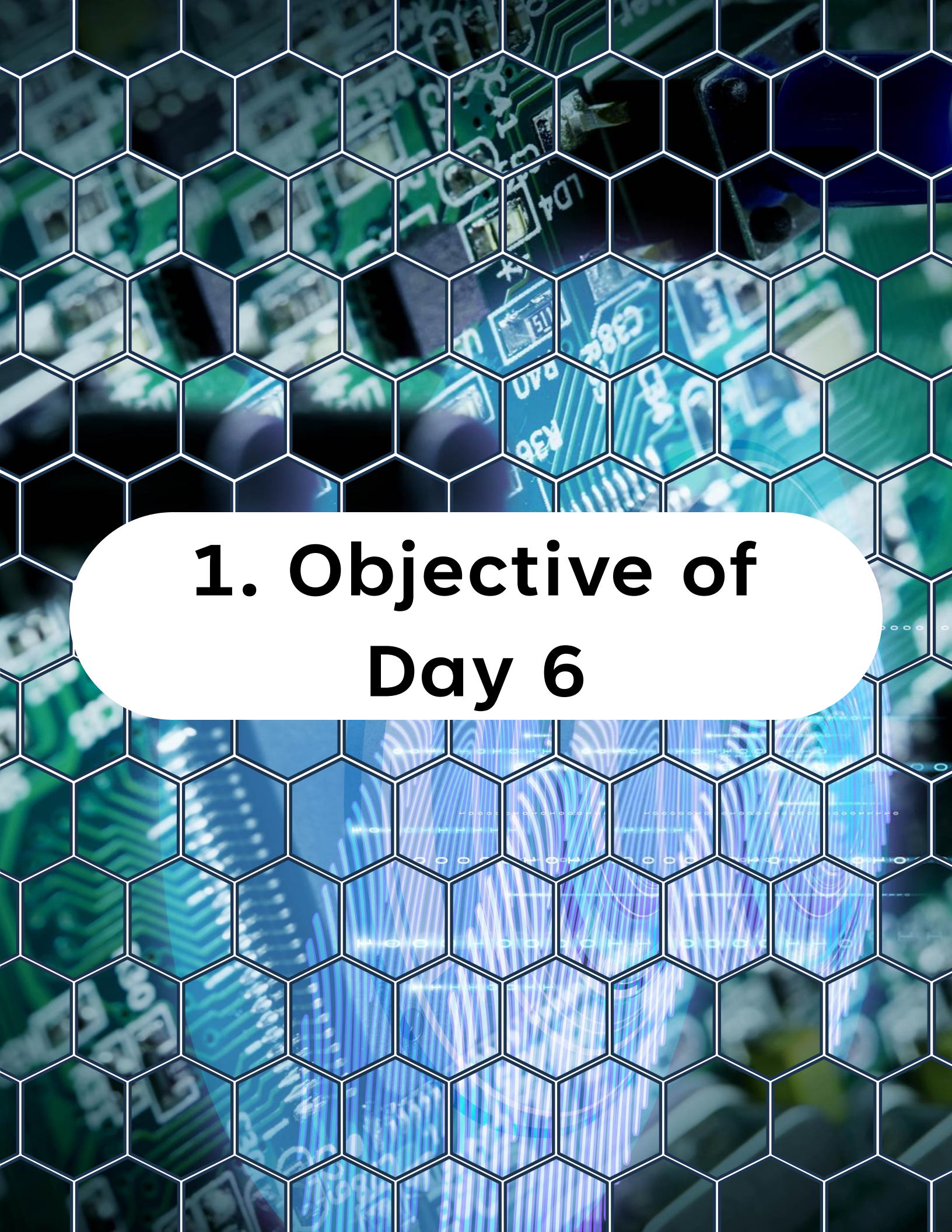
1. Objective of Day 6
2. Why Timers Matter
3. Timer Overview in AVR128DA48
4. Timer Counter A (**TCA**)
5. Timer Counter B (**TCB**)
6. Real-Time Counter (**RTC**)
7. Timer Clock Sources
8. Prescalers and Timer Resolution
9. Basic Timer Operating Modes
10. Timer Interrupt Concept
11. First Practical Example - Periodic Interrupt with TCB
12. How to Calculate Timer Values
13. Debugging Timers

Table of Contents

14.Using Timers as a System
Backbone

15.What You Learned Today

16.What Comes Next



1. Objective of Day 6

1. Objective of Day 6

Today you move from raw execution speed into controlled time.

Timers are the backbone of real embedded systems. They allow you to:

- Execute tasks periodically
- Measure time accurately
- Generate waveforms
- Trigger peripherals without CPU intervention

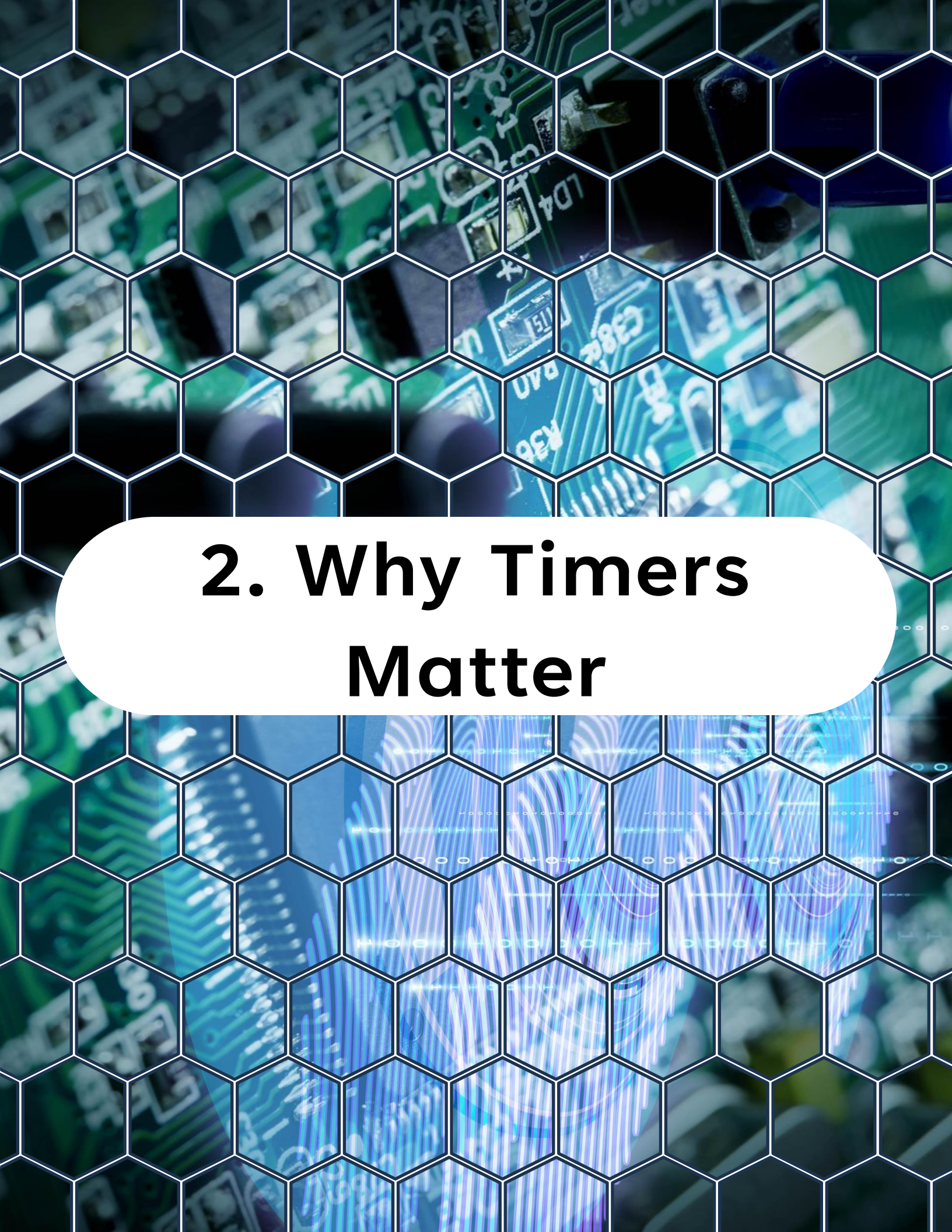
By the end of this day, you will:

- Understand the timer landscape of the AVR DA family
- Know when to use TCA, TCB, or RTC

1. Objective of Day 6

- Understand timer clock sources and prescalers
- Be able to configure a basic timer at register level
- Stop relying on delay loops forever

This is a major turning point in the course.



2. Why Timers Matter

2. Why Timers Matter

Without timers, firmware is blind to time.

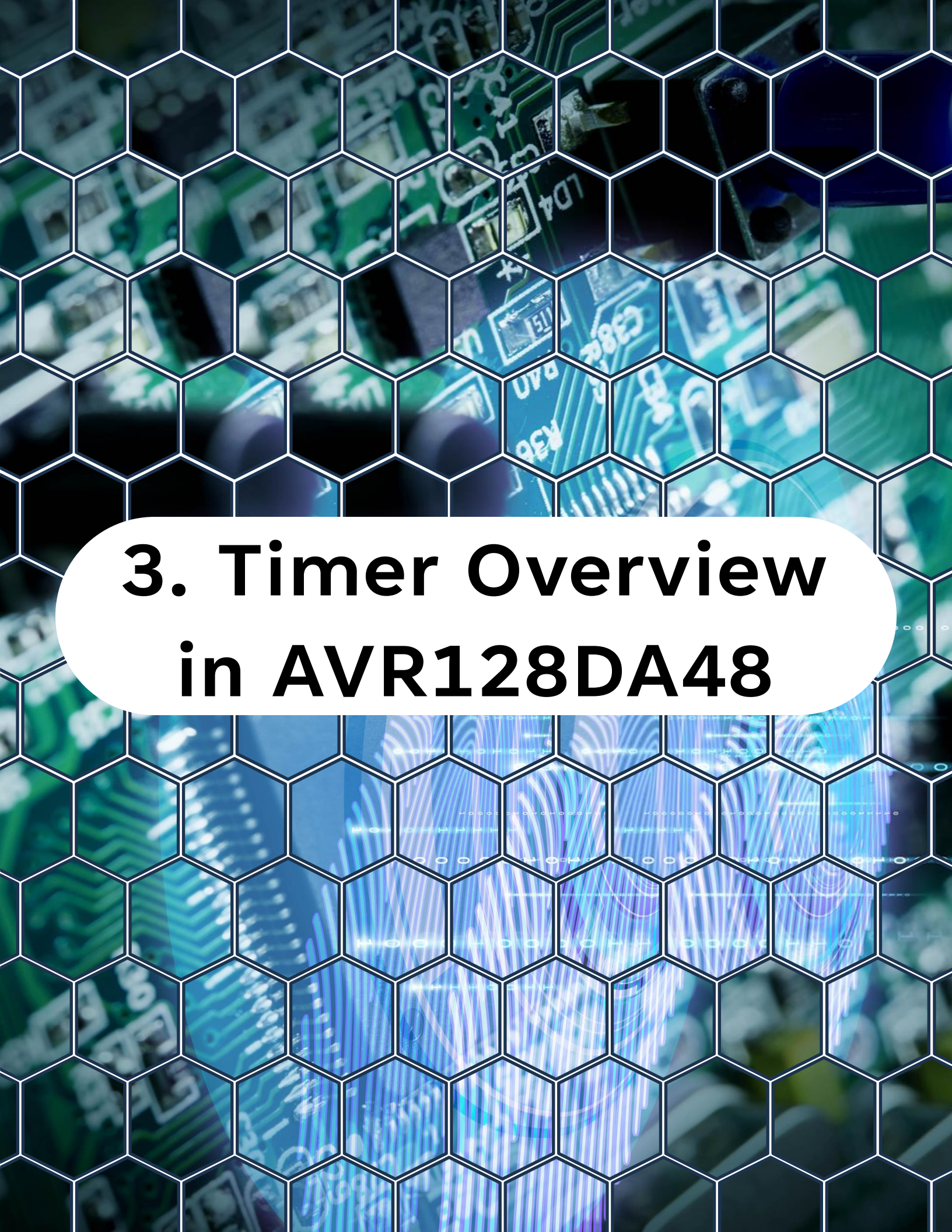
Delay loops:

- Waste CPU cycles
- Break when clock changes
- Do not scale
- Make systems fragile

Timers:

- Are hardware-based
- Are precise and repeatable
- Continue running during sleep
- Enable event-driven design

Professional embedded firmware is timer-driven, not delay-driven.



3. Timer Overview in AVR128DA48

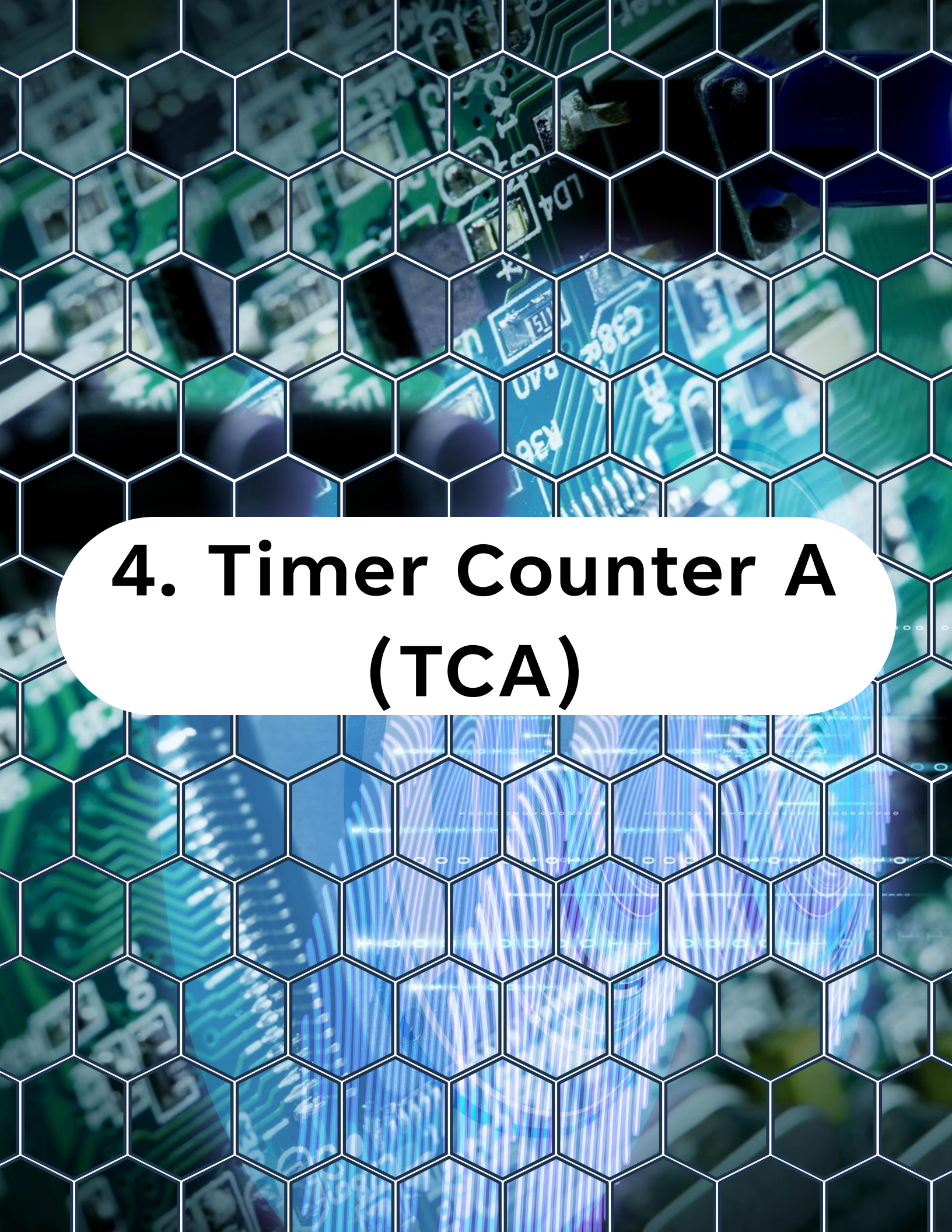
3. Timer Overview in AVR128DA48

The **AVR128DA48** provides multiple timer types, each optimized for a different role.

The main timers you will use are:

- TCA: Timer Counter A
- TCB: Timer Counter B
- RTC: Real-Time Counter

Each one exists for a reason. Using the wrong timer leads to messy code.



4. Timer Counter A (TCA)

4. Timer Counter A (TCA)

TCA is the most powerful and flexible timer.

Key characteristics:

- 16-bit resolution (or split into two 8-bit timers)
- Multiple compare channels
- PWM generation
- Periodic interrupts
- High-speed operation

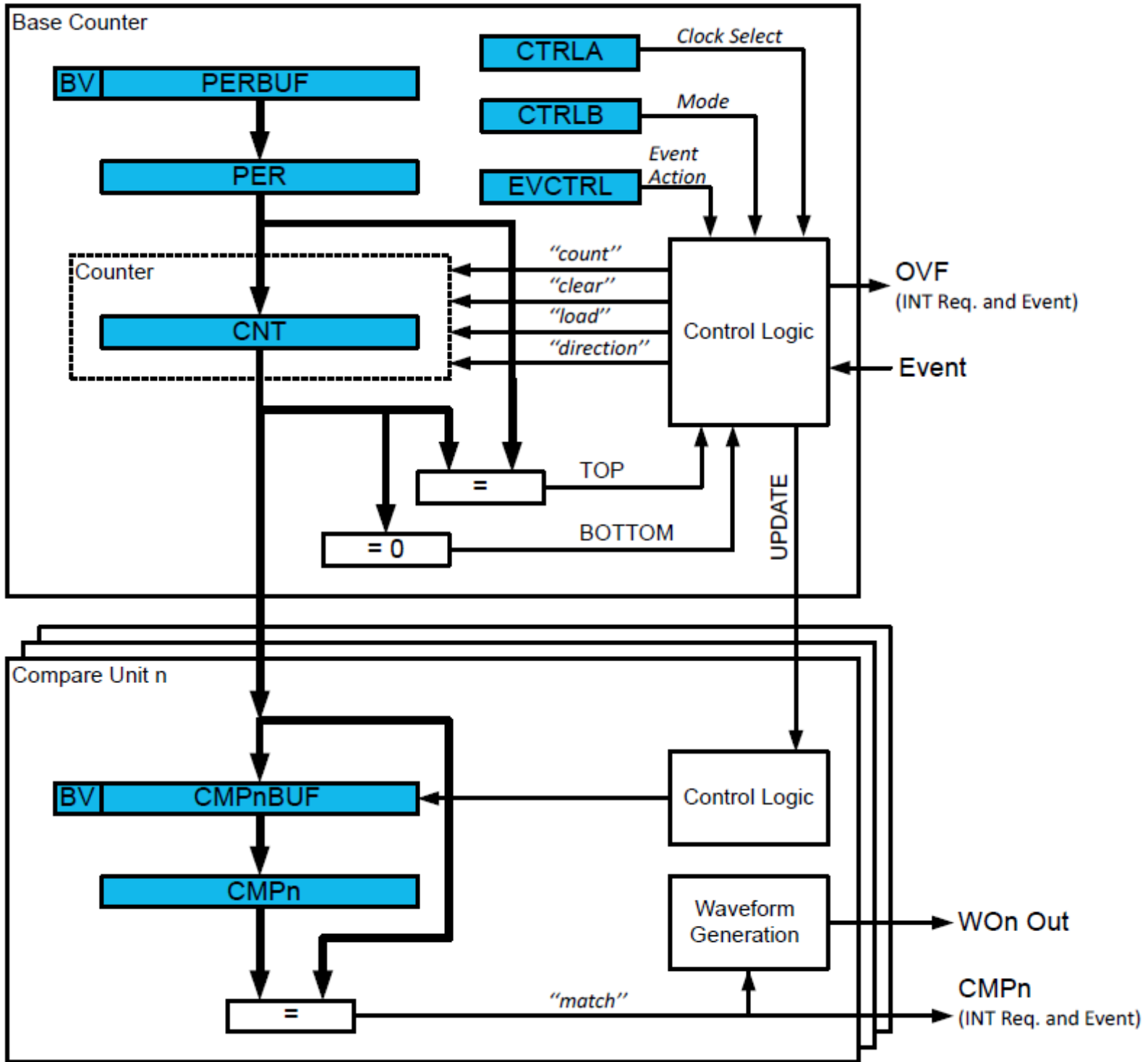
Typical use cases:

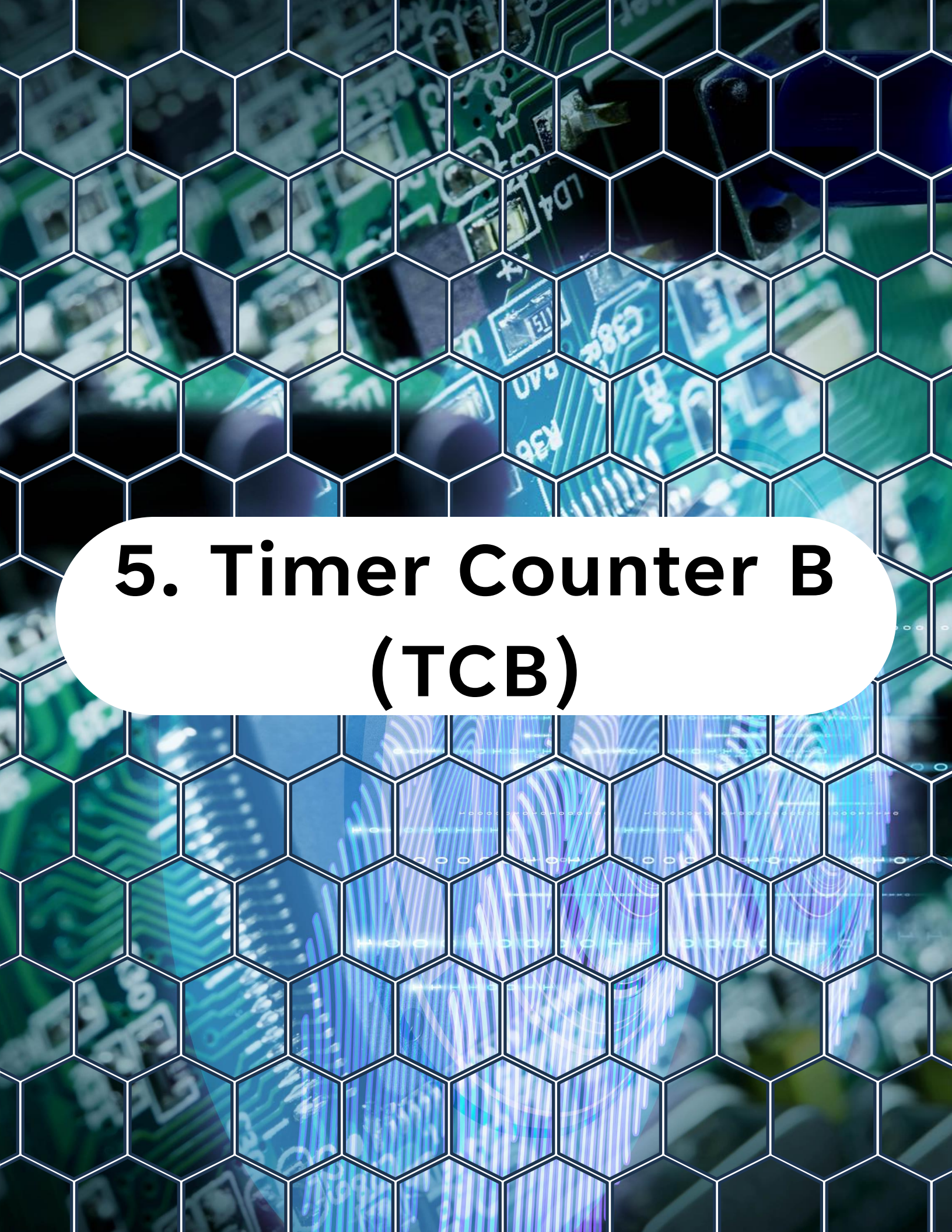
- PWM for LEDs, motors, power control
- System tick
- High-resolution timing

You will spend many days working with TCA.

4. Timer Counter A (TCA)

Timer/Counter Type A Block Diagram





5. Timer Counter B (TCB)

5. Timer Counter B (TCB)

TCB is simple, fast, and precise.

Key characteristics:

- 16-bit timer
- Single compare channel
- Low overhead
- Event system integration

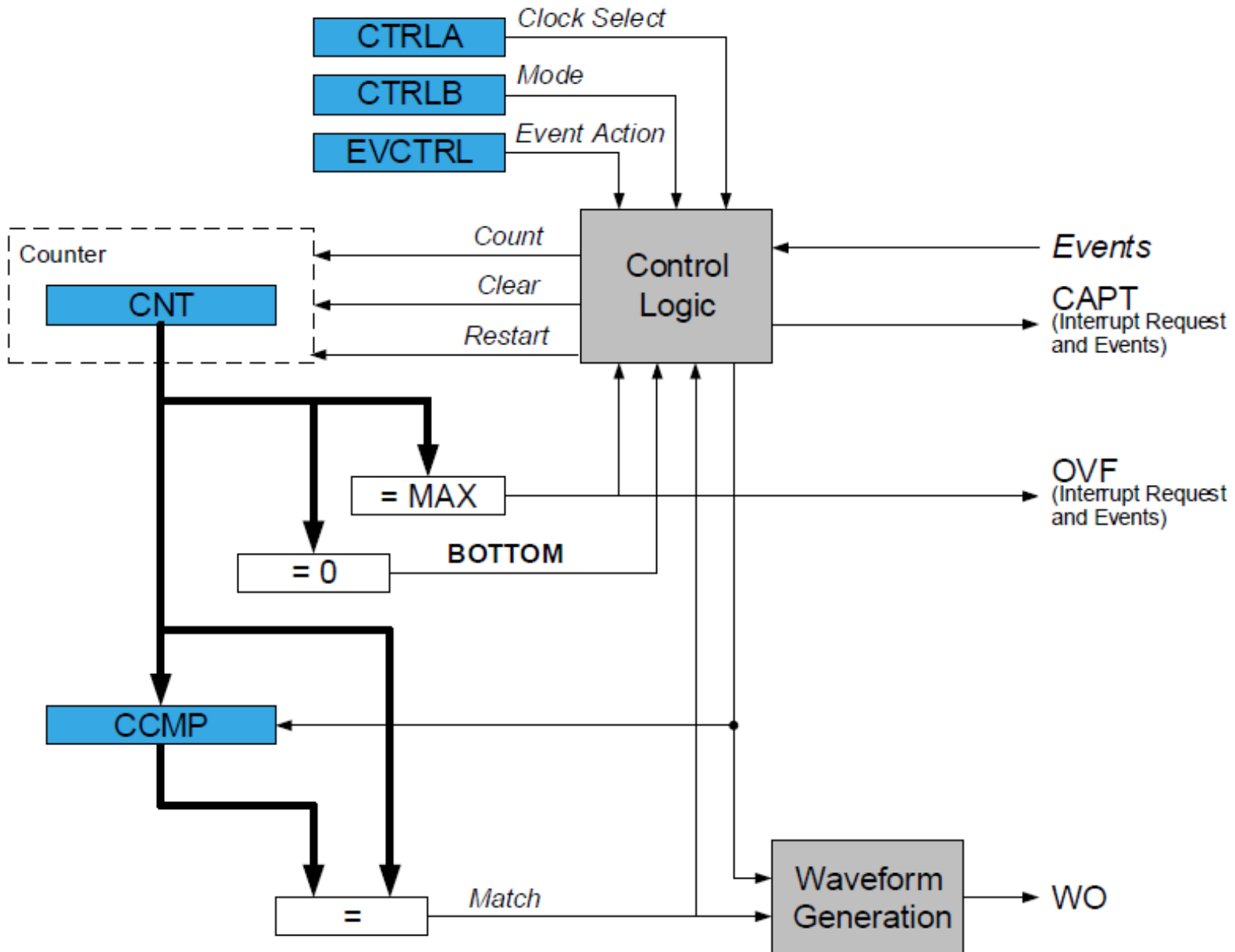
Typical use cases:

- Periodic interrupts
- One-shot delays
- Timeout supervision
- Lightweight scheduling

TCB is ideal when you want "just a timer" without complexity.

5. Timer Counter B (TCB)

Timer/Counter Type B Block Diagram





6. Real-Time Counter (RTC)

6. Real-Time Counter (RTC)

RTC lives in its own low-frequency clock domain.

Key characteristics:

- Usually driven by 32.768 kHz oscillator
- Extremely low power
- Runs during sleep
- Long time intervals

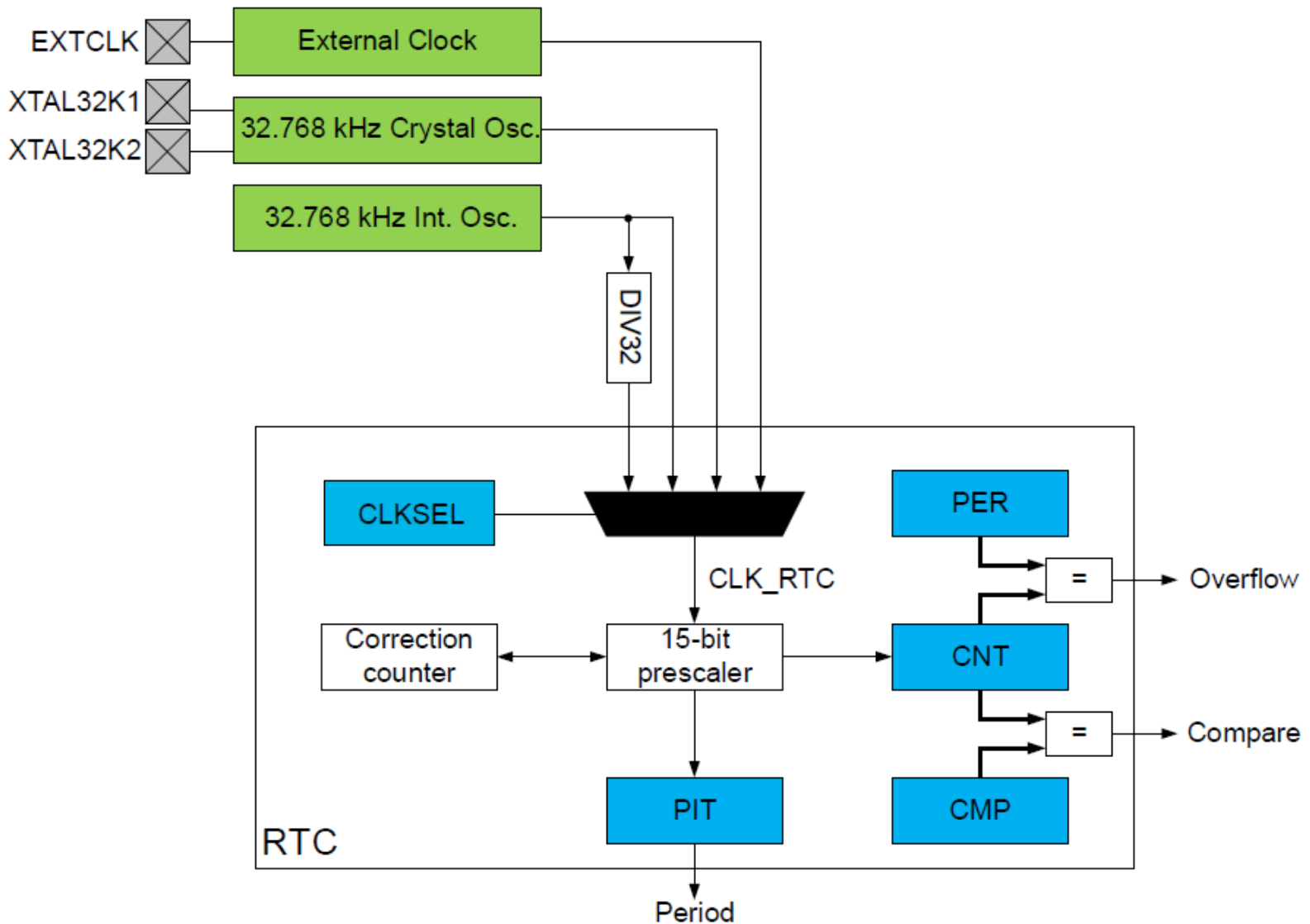
Typical use cases:

- 1-second ticks
- Long delays
- Timekeeping
- Wake-up from sleep

RTC is not for high-speed timing. It is for correctness and power efficiency.

6. Real-Time Counter (RTC)

RTC Block Diagram





7. Timer Clock Sources

7. Timer Clock Sources

Timers do not create time. They count clock ticks.

Possible clock sources:

- Main CPU clock
- Divided peripheral clock
- Low-frequency oscillator (RTC)

Every timer configuration must answer one question first:

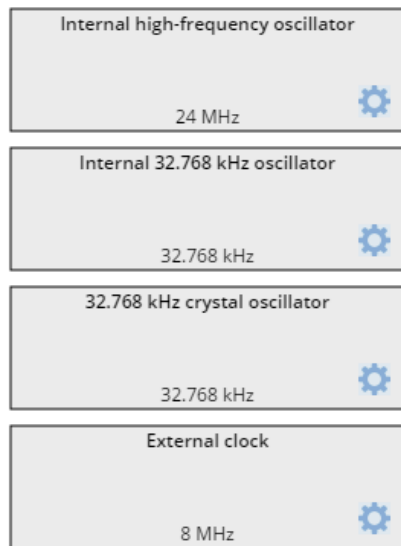
"What clock is this timer counting?"

If you skip this, everything else is wrong.

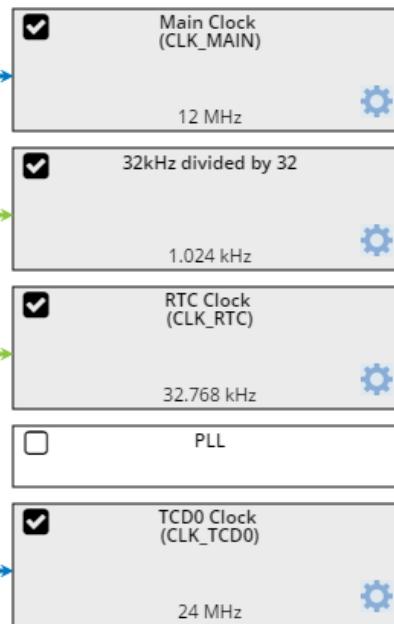
7. Timer Clock Sources

Possible Clock Sources Diagram

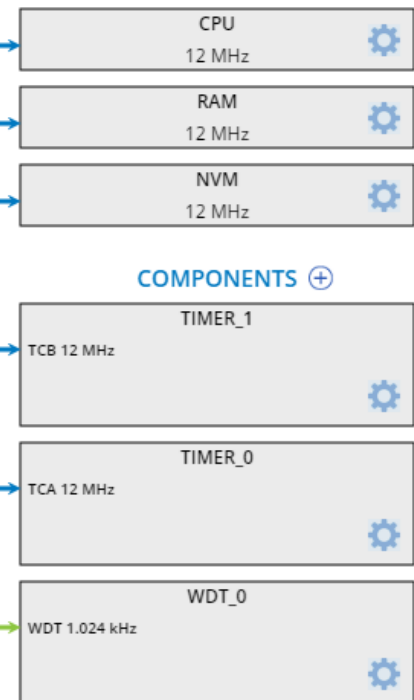
OSCILLATORS

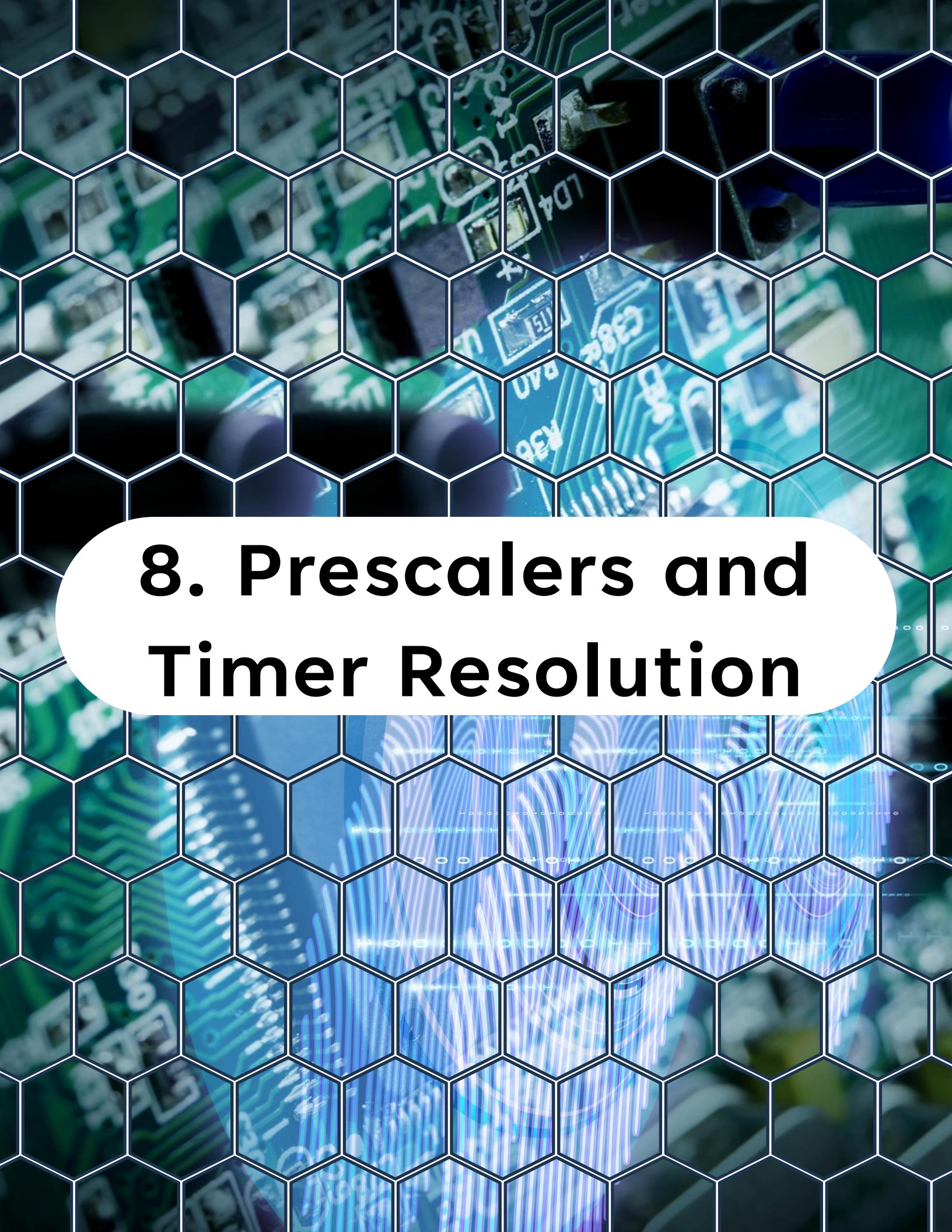


SOURCES



COMPONENTS ⊕





8. Prescalers and Timer Resolution

8. Prescalers and Timer Resolution

Prescalers divide the clock before it reaches the timer.

Example:

- CPU clock = 24 MHz
- Prescaler = 64
- Timer clock = 375 kHz

Tradeoffs:

- Lower prescaler -> higher resolution, shorter range
- Higher prescaler -> lower resolution, longer range

Timer design is always a balance between resolution and range.

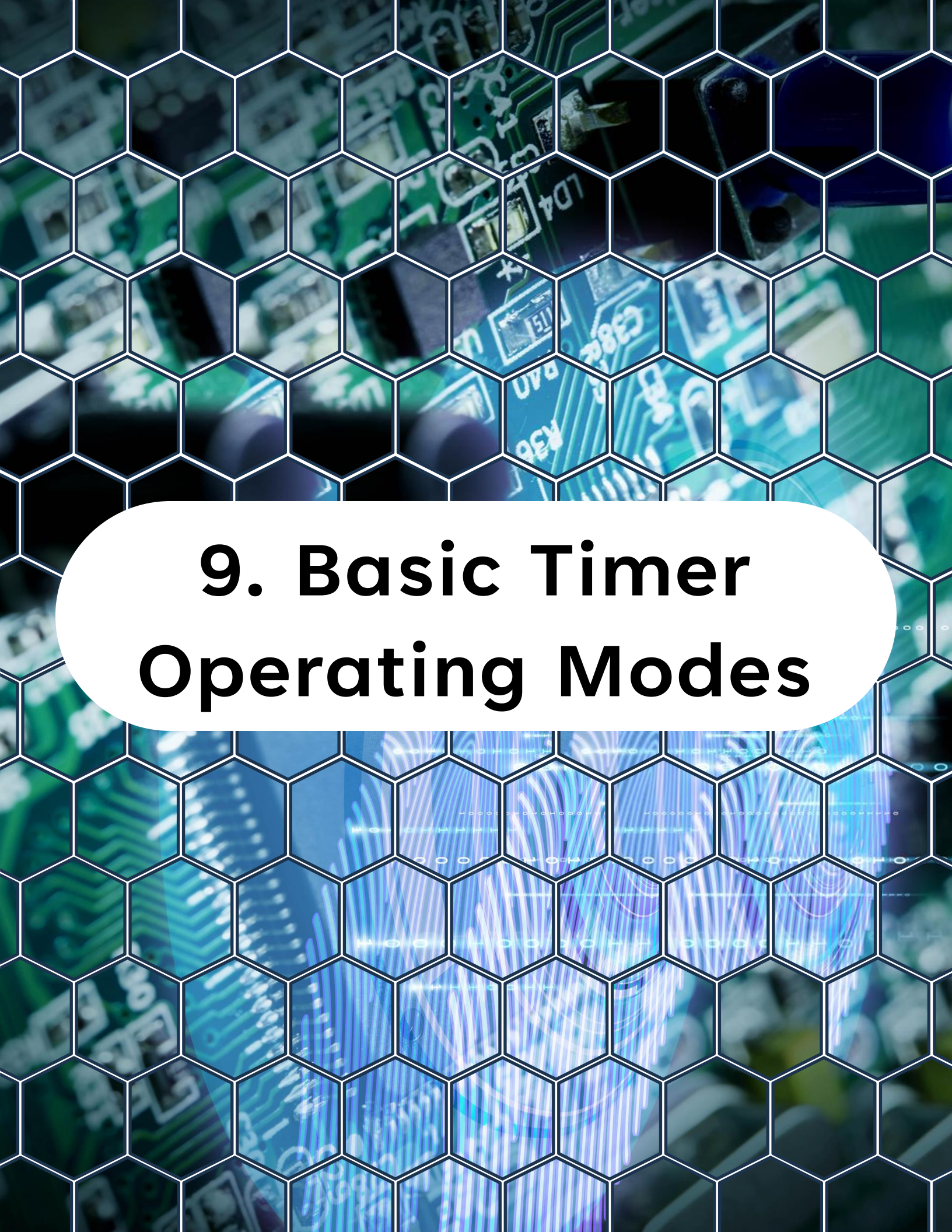
8. Prescalers and Timer Resolution

Clock Select

Bits 3:1 – CLKSEL[2:0] Clock Select

These bits select the clock frequency for the timer/counter.

Value	Name	Description
0x0	DIV1	$f_{TCA} = f_{CLK_PER}$
0x1	DIV2	$f_{TCA} = f_{CLK_PER}/2$
0x2	DIV4	$f_{TCA} = f_{CLK_PER}/4$
0x3	DIV8	$f_{TCA} = f_{CLK_PER}/8$
0x4	DIV16	$f_{TCA} = f_{CLK_PER}/16$
0x5	DIV64	$f_{TCA} = f_{CLK_PER}/64$
0x6	DIV256	$f_{TCA} = f_{CLK_PER}/256$
0x7	DIV1024	$f_{TCA} = f_{CLK_PER}/1024$



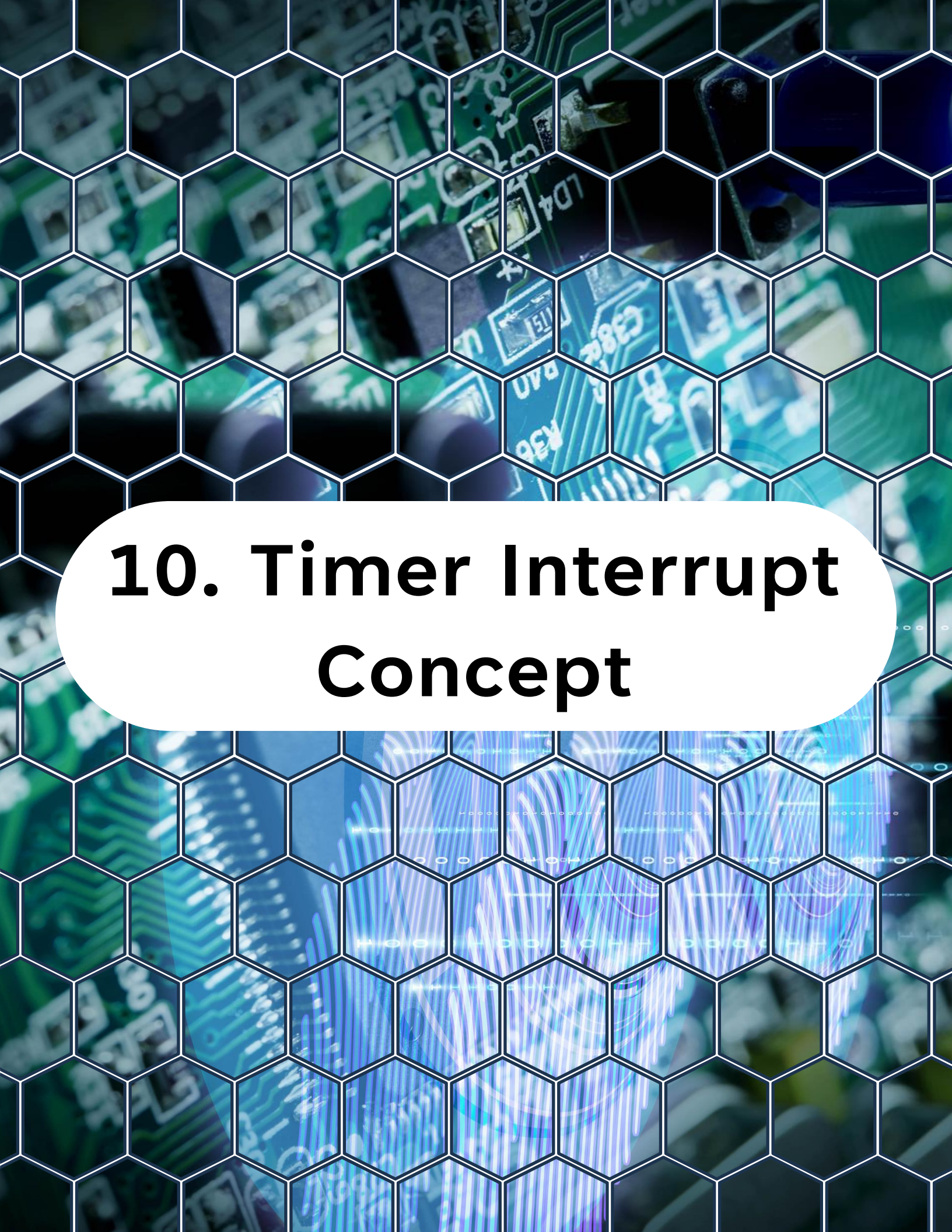
9. Basic Timer Operating Modes

9. Basic Timer Operating Modes

Most timers support some variation of:

- Normal mode (count up, overflow)
- Compare match mode
- Periodic interrupt mode
- PWM mode (for TCA)

You will start with the simplest: periodic interrupts.



10. Timer Interrupt Concept

10. Timer Interrupt Concept

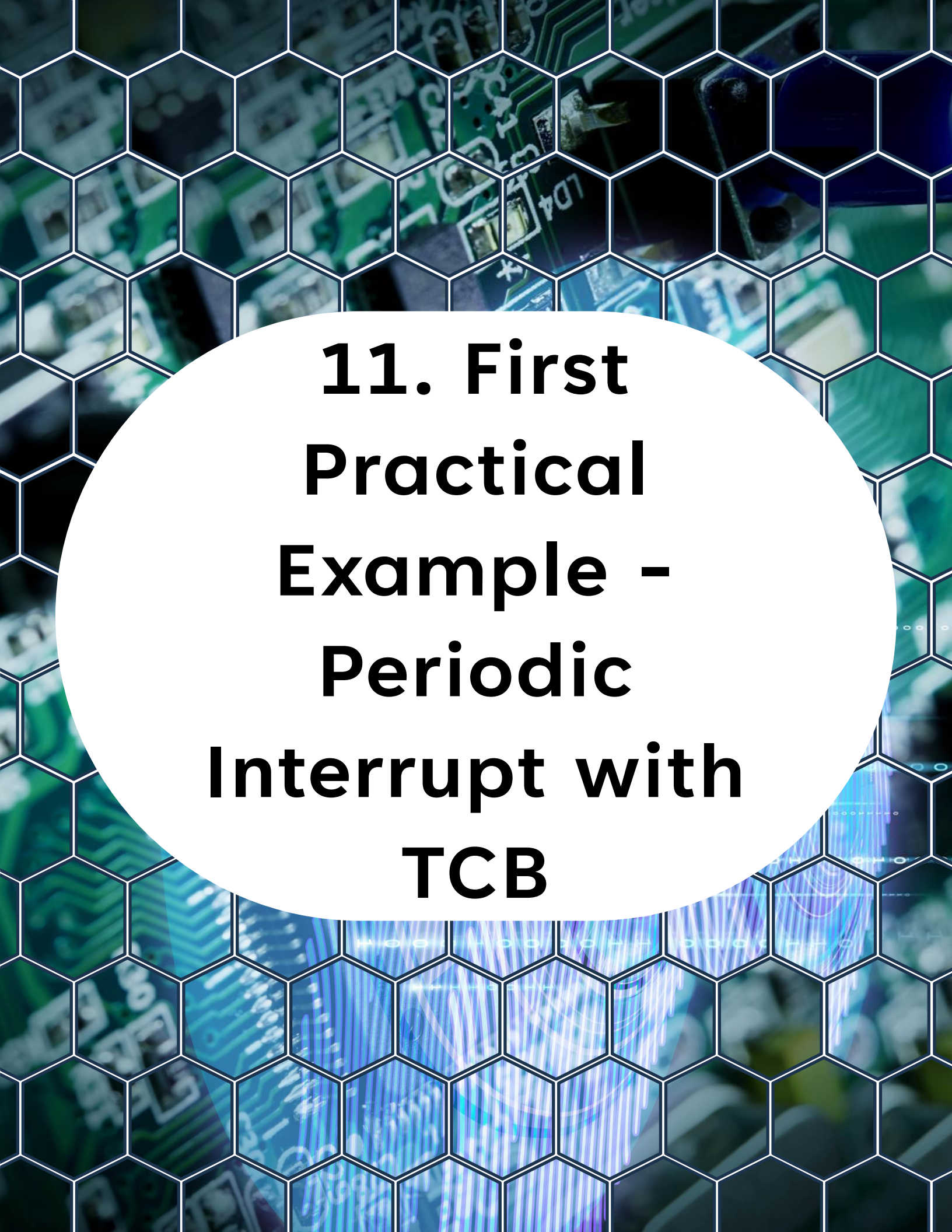
A timer interrupt occurs when:

- The timer reaches a defined value
- The timer overflows
- A compare match happens

The CPU:

- Finishes the current instruction
- Jumps to the ISR
- Executes your handler
- Returns to main code

This allows precise, non-blocking timing.



11. First Practical Example - Periodic Interrupt with TCB

11. First Practical Example - Periodic Interrupt with TCB

Goal

Generate a periodic interrupt using TCB without touching GPIO yet.

This example focuses only on timer configuration.

Step 1: Configure the System Clock



```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <stdint.h>
4
5  /*
6   * Configure system clock:
7   * - Select OSCHF as main clock source
8   * - Set OSCHF frequency to 24 MHz
9   * - Enable prescaler /2 -> CLK_PER = 12 MHz
10 */
11 static void clock_init_24mhz_presc2(void)
12 {
13     // Select main clock source: OSCHF, CLKOUT disabled
14     ccp_write_io((void*)&(CLKCTRL.MCLKCTRLA),
15                 CLKCTRL_CLKSEL_OSCHF_gc
16                 | 0 << CLKCTRL_CLKOUT_bp);
17
18     // Set OSCHF frequency to 24 MHz, AUTOTUNE disabled, RUNSTDBY disabled
19     ccp_write_io((void*)&(CLKCTRL.OSCCTRLA), CLKCTRL_FROSEL_24M_gc
```


11. First Practical Example - Periodic Interrupt with TCB

Step 1: Configure the System Clock

```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <stdint.h>
4
5  /*
6   * Configure system clock:
7   * - Select OSCHF as main clock source
8   * - Set OSCHF frequency to 24 MHz
9   * - Enable prescaler /2 -> CLK_PER = 12 MHz
10  */
11  static void clock_init_24mhz_presc2(void)
12  {
13      // Select main clock source: OSCHF, CLKOUT disabled
14      ccp_write_io((void*)&(CLKCTRL.MCLKCTRLA),
15                  CLKCTRL_CLKSEL_OSCHF_gc
16                  | 0 << CLKCTRL_CLKOUT_bp);
17
18      // Set OSCHF frequency to 24 MHz, AUTOTUNE disabled, RUNSTDBY disabled
19      ccp_write_io((void*)&(CLKCTRL.OSCHFCTRLA), CLKCTRL_FRQSEL_24M_gc
20                  | 0 << CLKCTRL_AUTOTUNE_bp
21                  | 0 << CLKCTRL_RUNSTDBY_bp);
22
23      // Enable main prescaler, divide by 2
24      ccp_write_io(&CLKCTRL.MCLKCTRLB, CLKCTRL_PEN_bm | CLKCTRL_PDIV_2X_gc);
25  }
```

11. First Practical Example - Periodic Interrupt with TCB

Step 2: Configure TCB in Periodic Interrupt Mode



```
1  /*
2   * Initialize TCB0 in Periodic Interrupt Mode for a 1 ms tick.
3   *
4   * With CLK_PER = 12 MHz and TCB clock = CLK_PER/2:
5   * - TCB clock = 6 MHz
6   * - CCMP = 6000 -> 1 ms period
7   */
8  static void tcb0_init_1ms_tick(void)
9  {
10     // Ensure timer is stopped before configuring
11     TCB0.CTRLA = 0;
12
13     // Period for 1 ms (6 MHz / 1000)
14     TCB0.CCMP = 6000;
15
16     // Periodic interrupt mode
17     TCB0.CTRLB = TCB_CNTMODE_INT_gc;
18
19     // Clear pending interrupt flag
20     TCB0.INTFLAGS = TCB_CAPT_bm;
21
22     // Enable interrupt
23     TCB0.INTCTRL = TCB_CAPT_bm;
24
25     // Start timer: clock = CLK_PER/2, enable
26     TCB0.CTRLA = TCB_CLKSEL_DIV2_gc | TCB_ENABLE_bm;
27 }
```

11. First Practical Example - Periodic Interrupt with TCB

Explanation:

- **CCMP** defines when the interrupt fires
- **CNTMODE** selects periodic interrupt mode
- **CLK_PER** means peripheral clock is used

Step 3: Define the ISR



```
1  /* TCB0 interrupt: fires every 1 ms */
2  ISR(TCB0_INT_vect)
3  {
4      g_ms_ticks++;
5      TCB0.INTFLAGS = TCB_CAPT_bm; // Clear interrupt flag
6  }
```

At this stage, the ISR increment the global 1 ms tick counter.

11. First Practical Example - Periodic Interrupt with TCB

Step 4: Initialize the Timer and Enable Global Interrupts

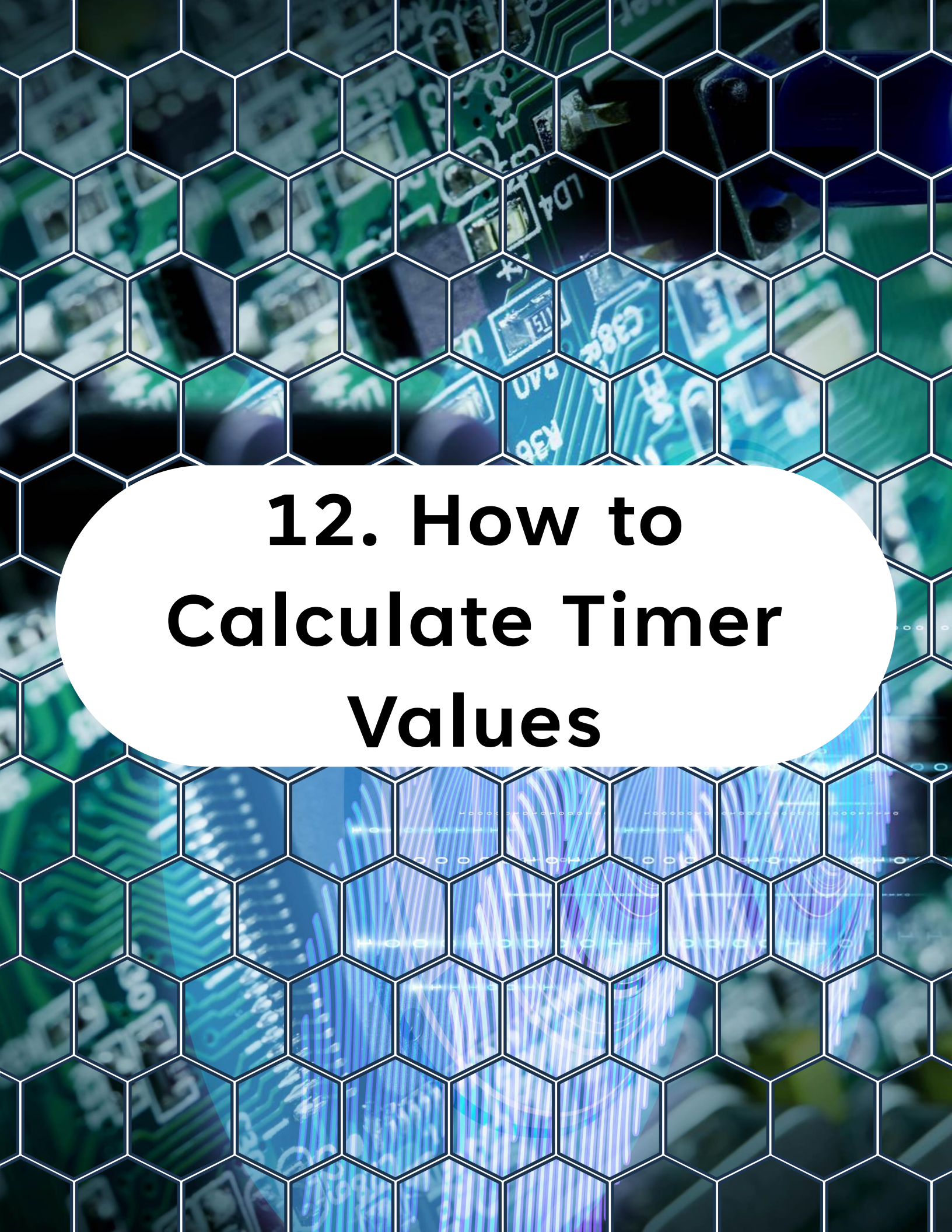
```
1 // Application entry point
2 int main(void)
3 {
4     // Configure system clock
5     clock_init_24mhz_presc2();
6
7     // Initialize TCB0 in Periodic Interrupt Mode for a 1 ms tick.
8     tcb0_init_1ms_tick();
9
10    // Enable global interrupts.
11    sei();
12
13    while (1)
14    {
15        /* Put a breakpoint and watch g_ms_ticks
16         incrementing every 1 ms */
17    }
18 }
```

11. First Practical Example - Periodic Interrupt with TCB

If this code runs:

- The CPU executes main()
- TCB generates interrupts periodically
- ISR executes silently in the background

You now have real time.



12. How to Calculate Timer Values

12. How to Calculate Timer Values

General formula:

**Timer ticks = Timer clock frequency / Desired
frequency**

Example:

- CPU clock = 24 MHz
- Peripheral clock = 24 MHz
- Desired interrupt = 1 kHz

Ticks = 24000

This is not guesswork. It is deterministic math.



13. Debugging Timers

13. Debugging Timers

Common timer mistakes:

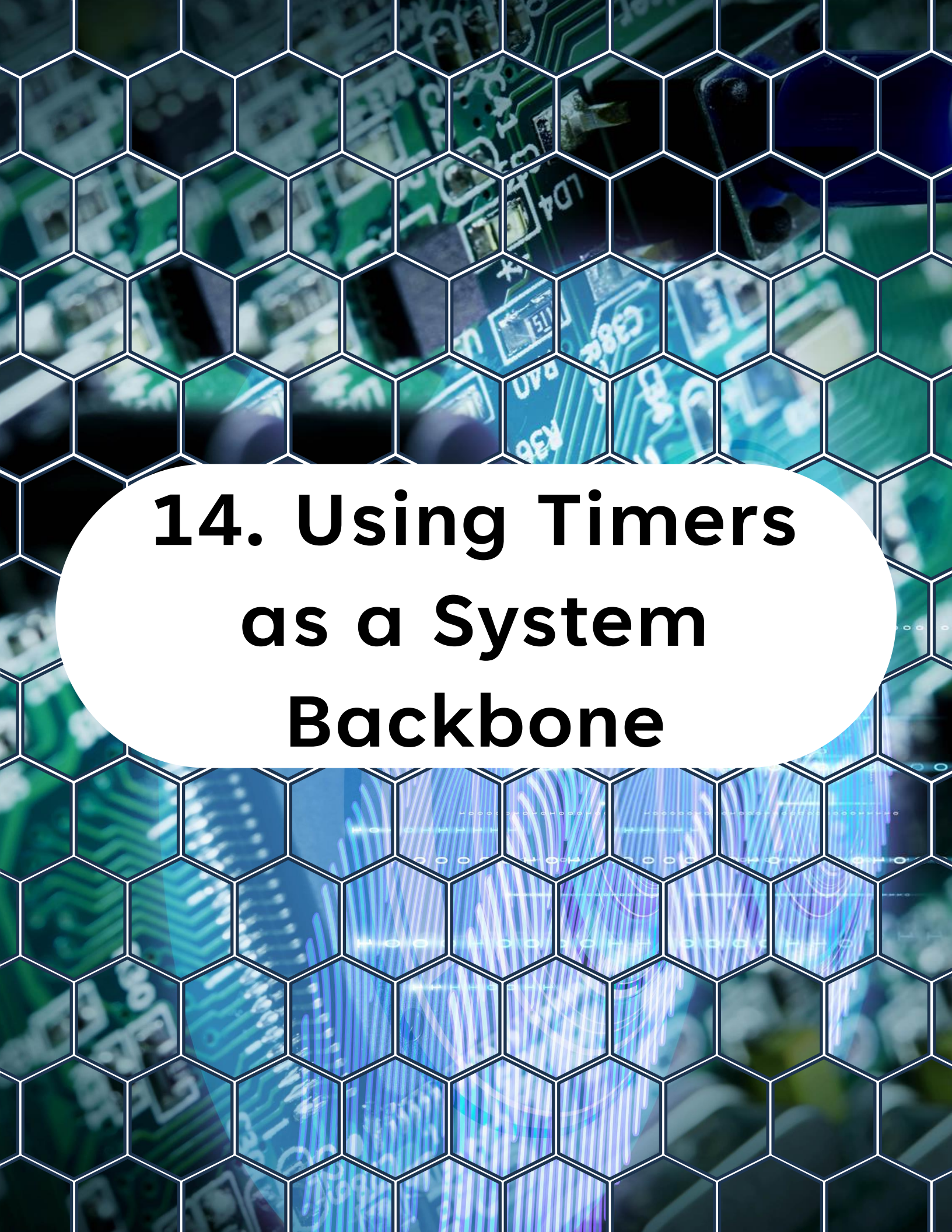
- Wrong clock source
- Prescaler forgotten
- Interrupt flag not cleared
- Interrupts not globally enabled
- Wrong ISR vector

Symptoms:

- ISR never runs
- ISR runs too fast or too slow
- System appears frozen

When debugging timers:

- Verify clock first
- Verify prescaler
- Verify interrupt enable
- Use GPIO toggles inside ISR



14. Using Timers as a System Backbone

14. Using Timers as a System Backbone

Timers enable:

- Cooperative schedulers
- Timeouts
- Periodic sampling
- State machine timing

From this point forward, timers will appear everywhere in this course.



15. What You Learned Today

15. What You Learned Today

By the end of Day 6, you understand:

- Why timers are essential
- The role of TCA, TCB, and RTC
- Timer clock sources and prescalers
- How timer interrupts work
- How to configure a basic timer at register level

You are no longer writing "while(1)" firmware.

You are building time-aware systems.



16. What Comes Next

16. What Comes Next

Day 7 - TCA Single Mode (PWM Basics)

Next, you will:

- Use TCA in single mode
- Generate real PWM signals
- Control duty cycle and frequency
- Drive LEDs with hardware timing

This is where timers start producing visible, useful output.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>